



دليل
مختصر

المصادقة والتفويض

JWT / OAuth



دليل مختصر لفهم المصادقة والتفويض
في الأنظمة الحديثة



مصادقة آمنة



تفويض دقيق



حماية البيانات



أداء عالي



متوافق مع المعايير

سلسلة شاملة تغطي



مفاهيم أساسية
واضحة



خطوات عملية
وتطبيقات



أفضل الممارسات
والأمان



أمثلة برمجية
حديثة



معمارية الأنظمة
وتكاملها



فهم المصادقة والتفويض باستخدام JWT و OAuth هو أساس بناء أنظمة آمنة وموثوقة وقابلة للتوسع.



..***
TOKEN

مفاهيم JWT / OAuth

دليل مختصر لفهم المصادقة والتفويض الحديث



OAuth 2.0

1 مقدمة المصادقة والتفويض

Authentication

المصادقة

التحقق من هوية المستخدم من أنت؟



Authorization

التفويض

تحديد ما يُسمح للمستخدم بفعله

ما الذي يُسمح لك به؟



Session vs Token

Session (مخزن)

- مخزن في
-
- يعتمد على الكوكيز

Token (حديث)

- مخزن في
- قابل للنقل والتوسع

2 JWT — JSON Web Token

ما هو JWT؟

هو معيار مفتوح (RFC 7519) لتمثيل معلومات بشكل آمن بين طرفين على هيئة Token.

بنية JWT

Header • Payload • Signature

1 Header (الرأس)

نوع الخوارزمية ونوع التوقيع

{ "alg": "HS256", "typ": "JWT" }

2 Payload (المحتوي)

البيانات (Claims)

{ "sub": "123", "name": "Ahmed", "role": "Admin", "exp": 1716800000, "iat": 1716796400 }

أشهر Claims

iss المصدر
sub المعرف
aud الجمهور
exp انتهاء الصلاحية
iat وقت الإصدار
nbf غير صالح قبل
jti معرف فريد

3 Signature (التوقيع)

توقيع مشفر للتحقق من عدم التلاعب

HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret)

أنواع التوكن



Access Token
صلاحية قصيرة
للوصول إلى الموارد



Refresh Token
صلاحية أطول
للحصول على توكن جديد

3 OAuth 2.0

ما هو OAuth؟

إطار تفويض يسمح للتطبيقات بالحصول على صلاحية الوصول إلى موارد المستخدم دون الحصول على بيانات اعتماد المستخدم.

الأطراف الرئيسية



Resource Owner
مالك المورد (المستخدم)



Client
التطبيق الذي يطلب الوصول



Authorization Server
خادم التفويض (المصادقة)



Resource Server
خادم الموارد المحمية

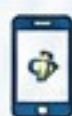


Access Token
رمز الوصول إلى الموارد

4 تدفقات OAuth 2.0 الشائعة



Authorization Code
الأكثر استخداماً في تطبيقات الويب
يتطلب إعادة توجيه المتصفح



Authorization Code + PKCE
للتطبيقات العامة (مثل الجوال)
حماية إضافية باستخدام PKCE



Client Credentials
(Machine to Machine)
بدون وجود مستخدم



Refresh Token
يستخدم للحصول على Access Token جديد
عند انتهاء صلاحية التوكن الحالي



Device Authorization
للأجهزة محدودة الإدخال (مثل Smart TV)
باستخدام كود تحقق

5 أمان JWT



توقيع Token
استخدم خوارزميات قوية مثل HS256 أو RS256



التحقق من الصلاحية exp
لا تقبل Token منتهية الصلاحية



التحقق من iss و aud
تأكد من الجهة المصدرة والجمهور المستهدف



لا تخزن أسرار داخل Payload
المحتوى قابل للقراءة بعد فك ترميز Base64



HTTPS دائماً
لمنع التنصت وسرقة التوكن

6 تخزين Tokens



HttpOnly Cookie
لا يمكن الوصول لها عبر JavaScript



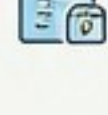
Secure Cookie
HTTPS عبر لتتابة



SameSite
يقلل من مخاطر CSRF



Local Storage
سهل الاستخدام لكن عرضة لـ XSS



Session Storage
مخزن مؤقت ينتهي بإغلاق التاب

7 دورة حياة Token



1 Login

المستخدم يسجل دخوله



2 Issue Token

الخادم يصدر Access Token و Refresh Token



3 Validate Token

الـ API يتحقق من صحة التوكن في كل طلب



4 Refresh Token

عند الانتهاء يستخدم Refresh للحصول على Token Access جديد



5 Revoke Token

إلغاء التوكن في الخادم (قائمة سوداء)



6 Logout

تسجيل الخروج وإزالة التوكن من العمل

8 أخطاء ومخاطر شائعة



Token Theft

سرقة التوكن من التخزين



CSRF

إعادة استخدام التوكن



Weak Secret

مفتاح ضعيف يسهل كسر التوقيع

XSS

تنفيذ سكريبتات خبيثة وسرقة التوكن

Replay Attack

إعادة استخدام التوكن

Long-lived Tokens

صلاحية طويلة تزيد المخاطر

9 JWT / OAuth مع REST API



Authorization Header

أرسل التوكن في كل طلب



Bearer Token

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5In0=



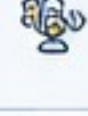
API Authentication

الـ API يتحقق من صحة التوكن



Role-Based Authorization

تحديد الصلاحيات بناءً على الأدوار



Policy-Based Authorization

سياسات مرنة للتحكم في الوصول

10 JWT / OAuth في ASP.NET Core



JWT Bearer Authentication

استخدم JwtBearer في Program.cs



Claims

User.Claims عبر Claims عبر فئات



Roles

استخدم الأدوار بها [Authorize(Roles="Admin")]



Policies

لحريف سياسات مخصصة في Authorization



[Authorize]

حماية التحكمات أو الـ Endpoints



Refresh Tokens

تخزين Refresh Token بشكل آمن وإدارته

11 مصطلحات سريعة

JWT

JSON Web Token

OAuth 2.0

معيار تبادل للوصفات بشكل آمن

OpenID Connect

طبقة هوية مبنية فوق OAuth 2

Access Token

يمنح وصولاً للموارد لفترة محددة

Refresh Token

يستخدم للحصول على Access Token جديد

Scope

نطاق الصلاحيات المطلوبة

Claim

معلومة عن المستخدم داخل التوكن

Bearer

نوع التوكن المرسل في الطلب (Bearer)

PKCE

(Proof Key for Code Exchange)

12 مثال معماري مختصر



ملاحظات سريعة

- ✓ استخدم صلاحيات قصيرة للتوكن (Access Token).
- ✓ استخدم Refresh Token بأمان وشكله جيداً.
- ✓ تحقق من جميع Claims الحساسة.
- ✓ احفظ مفاتيح التوقيع بأمان تام.
- ✓ راقب وسجل الأحداث الأمنية.



المصادقة والتفويض JWT / OAuth



TOKEN



دليل مختصر لفهم المصادقة والتفويض في الأنظمة الحديثة

1. مقدمة المصادقة والتفويض

هما أساس أمان أي نظام رقمي، يضمن التحقق من هوية المستخدم وتحديد ما يمكنه القيام به.

لماذا هما مهمان؟

- ✓ حماية البيانات والموارد من الوصول غير المصرح به.
- ✓ ضمان وصول المستخدمين المصرح لهم فقط.
- ✓ تعزيز ثقة المستخدمين في النظام.
- ✓ امتثال للمعايير الأمنية واللوائح.

ما هي المصادقة؟ (Authentication)

المصادقة هي عملية التحقق من هوية المستخدم للتأكد من أنه الشخص الصحيح.

تجيب على السؤال:

من أنت؟



أمثلة لطرق المصادقة:



اسم مستخدم
وكلمة مرور



رمز تحقق
(OTP)



بصمة الإصبع



التعرف على
الوجه

ما هو التفويض؟ (Authorization)

التفويض هو تحديد ما يمكن للمستخدم المصرح له القيام به داخل النظام.

تجيب على السؤال:

ماذا يحق لك أن تفعل؟



أمثلة على التفويض:



قراءة البيانات



تعديل البيانات



حذف البيانات



إدارة إعدادات
النظام

العلاقة بين المصادقة والتفويض

1. المصادقة

تحقق من هوية المستخدم
(تسجيل الدخول).

من أنت؟



2. التفويض

تحديد صلاحيات المستخدم
وسماح الوصول للموارد.

ماذا يحق لك أن تفعل؟



3. الوصول الآمن

الوصول فقط للموارد
المسموح بها.



كيف تساعد JWT و OAuth

JWT

JWT (JSON Web Token)

- ✓ آلية لتغليف المعلومات (الهوية والصلاحيات) في Token آمن.
- ✓ يُرسل مع كل طلب للتحقق من الهوية والصلاحيات.
- ✓ لا يحتاج إلى تخزين جلسة في الخادم (Stateless).
- ✓ سريع وخفيف ومناسب لتطبيقات API و الموبايل.

header.payload.signature

2

OAuth 2.0

- ✓ إطار عمل لتفويض آمن يسمح للتطبيقات بالوصول إلى موارد المستخدم دون مشاركة كلمة المرور.
- ✓ يستخدم Grant Types مختلفة حسب نوع التطبيق.
- ✓ يعتمد على Access Token للوصول إلى الموارد.
- ✓ يستخدم على نطاق واسع في التكامل مع خدمات خارجية مثل Google, Microsoft, GitHub وغيرها.

أين يُستخدمان؟



تطبيقات الويب
و API



تطبيقات الجوال



التكامل مع
الخدمات الخارجية



التجارة الإلكترونية
و المنصات



الأنظمة المؤسسية



إنترنت الأشياء
(IoT)



فهم المصادقة والتفويض باستخدام JWT و OAuth هو أساس بناء أنظمة آمنة وموثوقة وقابلة للتوسع.





المصادقة والتفويض JWT / OAuth

دليل مختصر لفهم المصادقة والتفويض في الأنظمة الحديثة



2. JWT – JSON Web Token

هي طريقة آمنة ومفتوحة للبادل المعلومات بين طرفين كـ JS يتم نوقبها للتحقق من صحة وعدم تعديله،
• ويتكون من ثلاث أجزاء مفصولة بنقطة (.)

بنية JWT

Header

الرأس

Payload

المحتوى (البيانات)

Signature

التوقيع

يتم فصل الأجزاء الثلاثة
بنقطة (.)

xxxxxx.yyyyyy.zzzzzz

1 Header (الرأس)

يحتوي على نوع الترميز وخوارزمية التوقيع.

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

أمثلة للخوارزميات:

HS256 • HS512 • RS256 • ES256

2 Payload (المحتوى)

يحتوي على مجموعة من البيانات (Claims).
هناك نوعان من الـ Claims:

Registered (المقاييسية Claims)

- iss : المُصدر (Issuer)
- sub : الموضوع (Subject)
- aud : الجمهور (Audience)
- exp : وقت انتهاء الصلاحية (Expiration Time)
- nbf : لا يُستخدم قبل (Not Before)
- iat : وقت الإصدار (Issued At)
- jti : معرف فريد (JWT ID)

Private Claims (الخاصة)

يمكن تعريف أي بيانات إضافية حسب
احتياج التطبيق.

```
{
  "sub": "12345",
  "name": "Ahmed",
  "role": "Admin",
  "email": "ahmed@example.com",
  "iat": 1715654000,
  "exp": 1715667600
}
```



الوقت يكون بصيغة Unix Timestamp

3 Signature (التوقيع)

يتم إنشاء التوقيع باستخدام
Header و Payload و معًا،
و سري (Secret) للتحقق من صحة Token.

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  secret
)
```

وظيفة التوقيع:



- التحقق من عدم تعديل البيانات.
- التأكد من هوية المُصدر.
- ضمان عدم التلاعب بالمحتوى.

مثال على JWT كامل



Header
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9

Payload
eyJzdWIiOiIxMjM0NTY3ODFiIiwiaWF0IjoxNjE1MjM0MDAwLCJleHAiOiJlbnR5cCI6IkpXVCJ9

Signature
SfIKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

JWT
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODFiIiwiaWF0IjoxNjE1MjM0MDAwLCJleHAiOiJlbnR5cCI6IkpXVCJ9.SfIKxwRJSMeKKF2QT4fwpMeJf36POk6y

أنواع Tokens المستخدمة مع JWT



Access Token

يستخدم للوصول إلى الموارد المحمية.
يكون قصير العمر (عادة دقائق).
يحتوي على الصلاحيات (Scopes / Roles).



Refresh Token

يستخدم للحصول على Access Token جديد.
يكون طويل العمر (أيام أو أسابيع).
يُحفظ في مكان آمن.

مزايا استخدام JWT



Stateless (خالي من الحالة)

لا يحتاج الخادم لتخزين الجلسات.



قابل للتوسع

مثالي للأنظمة الموزعة والميكروسيرفيس.



آمن

يتم التحقق من صحة التوقيع والمحتوى.



تجربة موحدة

يدعم الويب، الجوال، وتطبيقات سطح المكتب API.



أداء عالي

حجم صغير وسريع في النقل والمعالجة.

المفاهيم الأساسية



المستخدم

يقوم بتسجيل الدخول



العميل (Client)

يرسل بيانات الاعتماد



خادم المصادقة
(Authorization Server)

يتحقق من البيانات
ويصدر JWT.



خادم الموارد
(Resource Server)

يتحقق من صحة الـ JWT.
قبل السماح بالوصول



الموارد

يتم الوصول إلى
البيانات والخدمات

نصائح سريعة



استخدم خوارزميات توقيع قوية
مثل HS256 أو RS256.



اجعل مدة صلاحية Access Token قصيرة.



استخدم Refresh Token بشكل آمن.



لا تخزن معلومات حساسة
في Payload.



استخدم HTTPS لنقل Tokens.



(exp, iss, aud, signature).
عند استقبال الـ JWT.



المصادقة والتفويض JWT / OAuth

دليل مختصر لفهم المصادقة والتفويض في الأنظمة الحديثة

TOKEN



3. OAuth 2.0

OAuth 2.0 هو إطار عمل تفويض (Authorization Framework) يتيح لتطبيق ما الوصول إلى موارد محمية نيابة عن مستخدم، دون مشاركة بيانات اعتماد المستخدم.

ما هو التفويض؟

بدلاً من مشاركة اسم المستخدم وكلمة المرور مع كل تطبيق، يقوم OAuth بمنح التطبيق صلاحية محددة للوصول إلى الموارد نيابة عنك.

الأطراف الرئيسية في OAuth 2.0



1. مالك المورد
(Resource Owner)

المستخدم الذي يملك الموارد ويحكم في منح التطبيق صلاحية الوصول إليها.



2. العميل
(Client)

التطبيق الذي يطلب الوصول إلى موارد محمية نيابة عن المستخدم.



3. خادم التفويض
(Authorization Server)

يصادق المستخدم ويصدر Tokens (رموز الوصول) للعميل بعد الحصول على موافقته.



4. خادم الموارد
(Resource Server)

الخادم الذي يستضيف الموارد المحمية ويتحقق من صحة Token قبل منح الوصول.



Token (رمز الوصول)
(Access Token)

رمز يمثل تفويضاً يمنح العميل صلاحية محددة للوصول إلى الموارد نيابة عن المستخدم.

كيف يعمل OAuth 2.0 ؟ (تدفق مبسط)



خصائص OAuth 2.0



تفويض وليس مصادقة

لا يتعامل OAuth مع تسجيل دخول المستخدم. بل مع منح الصلاحيات.



صلاحيات محددة (Scopes)

يمنح الوصول فقط للموارد أو العمليات المسموح بها.



رموز قصيرة العمر

Tokens مؤقتة قابلة للتجديد أو الإبطال. لزيادة الأمان.



قابلية التوسع والتكامل

يعمل عبر تطبيقات ومؤسسات مختلفة بسهولة.

أنواع الرموز (Tokens)



Access Token

يستخدم للوصول إلى الموارد المحمية. قصير العمر. يحتوي على Scopes تحدد الصلاحيات.



Refresh Token

يستخدم للحصول على Access Token جديد. طويل العمر نسبياً. يجب حفظه بأمان شديد.

أمثلة استخدام OAuth 2.0



Google / Facebook عبر المصادقة باستخدام "تسجيل الدخول عبر..."



منح تطبيق طرف ثالث (مثل تطبيق جوال) الوصول إلى بياناتك في خدمة ما.



تكامل الخدمات بين الأنظمة (API Integration).



التفويض في تطبيقات الشركات والمؤسسات.

فوائد OAuth 2.0



- يوفر وصولاً آمناً دون مشاركة بيانات الاعتماد.
- يقلل من المخاطر عند استخدام تطبيقات خارجية.
- يمنح المستخدم تحكماً في الصلاحيات.
- يدعم التكامل بين الأنظمة والخدمات بسهولة.
- قابل للتوسع ويدعم سيناريوهات متعددة.

أمثلة على Scopes شائعة

Scope	الوصف
profile	معلومات الملف الشخصي.
email	الوصول إلى البريد الإلكتروني.
read	قراءة البيانات.
write	إنشاء أو تعديل البيانات.
delete	حذف البيانات.
offline_access	الوصول بدون اتصال (باستخدام Refresh Token).

ملاحظات مهمة

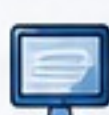
- استخدم دائماً HTTPS لحماية البيانات.
- لا تُخزن Access Token لفترة طويلة أكثر من اللازم.
- لا تطلب صلاحيات (Scopes) كبيرة إلا عند الحاجة.
- احفظ Refresh Token بأمان شديد ولا تشاركه.

مصطلحات سريعة



Authorization

تفويض الوصول إلى الموارد.



Client

التطبيق الذي يطلب الوصول للموارد.



Authorization Server

Tokens الخادم الذي يصدر بعد تفويض المستخدم.



Resource Server

الخادم الذي يحمي الموارد ويتحقق من Token.



Token

رمز بمنح صلاحية الوصول للموارد.



Scope

تحديد نطاق الصلاحيات الممنوحة.



المصادقة والتفويض

JWT / OAuth

دليل مختصر لفهم المصادقة والتفويض في الأنظمة الحديثة



4. OAuth 2.0 Flows

طرق التفويض في OAuth 2.0

تحدد طريقة ال Flow الأنسب للتطبيق بناءً على نوع العميل (Client) والسياق الأمني للاستخدام.

ما هي ال Flow ؟

هي خطوات تسير عليها عملية التفويض بين العميل ومخدم التفويض للحصول على Access Token آمن.

المكونات الرئيسية



مالك المورد
(Resource Owner)
المستخدم الذي يمنح الإذن.



العميل (Client)
التطبيق الذي يطلب الوصول إلى الموارد.



مخدم التفويض
(Authorization Server)
يُصدر الرموز (Tokens) ويتحقق من هوية العميل والمستخدم.



مخدم الموارد
(Resource Server)
يحتوي على الموارد المحمية.

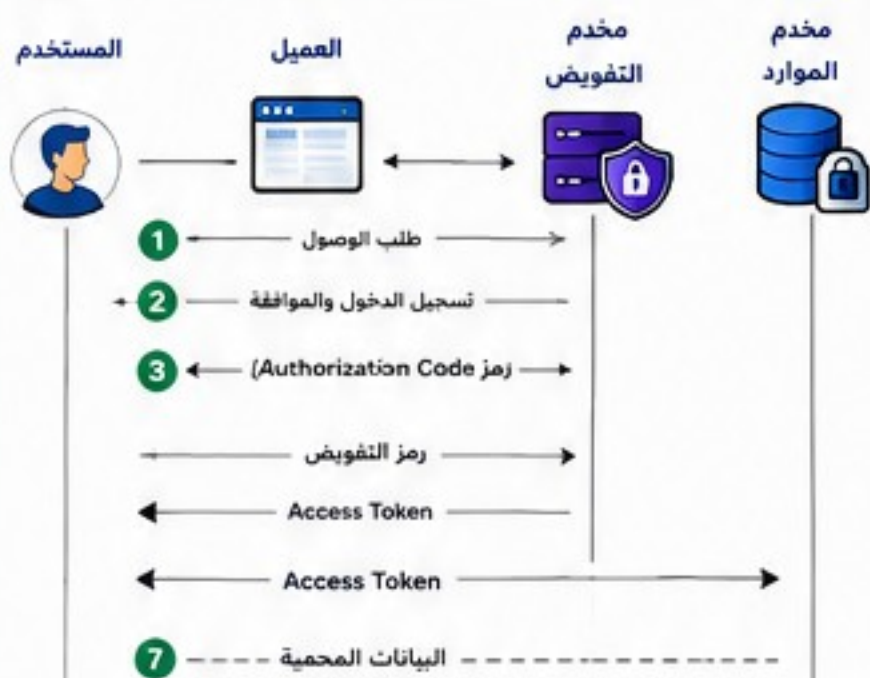


رمز الوصول (Access Token)
يُستخدم للوصول إلى الموارد المحمية.

طرق OAuth 2.0 الشائعة

1 Authorization Code Flow

تدفق رمز التفويض (الأكثر أماناً لتطبيقات الويب)



متى يُستخدم؟

لتطبيقات الويب التي تعمل على خادم خاص (Back-end)

المزايا

- ✓ آمن جداً
- ✓ Refresh Token
- ✓ مناسب لمعظم تطبيقات الويب

2 Authorization Code Flow + PKCE

رمز التفويض مع PKCE (مفسر SPA)



متى يُستخدم؟

لتطبيقات الجوال والتطبيقات أحادية الصفحة (SPA).

المزايا

- ✓ آمن لتطبيقات الجوال
- ✓ يمنع هجمات اعتراض رمز التفويض
- ✓ Refresh Token

3 Client Credentials Flow



متى يُستخدم؟

لتطبيقات تتفاعل بين الخوادم (Machine to Machine)

المزايا

- ✓ لا يحتاج لمستخدم
- ✓ سريع وبسيط
- ✓ مناسب للتكاملات الخلفية (APIs)

4 Refresh Token Flow

تدفق رمز التحديث



متى يُستخدم؟

لتجديد Access Token بدون إعادة تسجيل الدخول.

المزايا

- ✓ تحسين تجربة المستخدم
- ✓ يحافظ على الجلسة
- ✓ آمن مع إدارة صحيحة لـ Refresh Token

5 Device Authorization Flow

تدفق تفويض الجهاز (للأجهزة ذات الشاشات المحدودة)



متى يُستخدم؟

للأجهزة مثل التلفاز، أجهزة الألعاب، وأجهزة إنترنت الأشياء.

المزايا

- ✓ لا يحتاج لوحة مفاتيح
- ✓ تجربة سهلة للمستخدم
- ✓ آمن ومرن

مقارنة سريعة

Flow ال	الحالة	نوع العميل المناسب	دعم Refresh Token
Authorization Code	آمن جداً	تطبيقات الويب (خادم خاص)	✓
Authorization Code + PKCE	آمن	تطبيقات الجوال و ال SPA	✓
Client Credentials	بدون مستخدم	تطبيقات خادم إلى خادم	✗
Refresh Token	يعتمد على Flow	تجديد الجلسة	✓
Device Authorization	مناسب للأجهزة	أجهزة محدودة الإدخال	✓

نصائح مهمة

- ✓ استخدم دائماً HTTPS لحماية البيانات أثناء انتقالها.
- ✓ قم بتخزين Access Token بأمان (مثل Secure HttpOnly أو Secure Storage).
- ✓ جدد صلاحيات الوصول باستخدام Scopes المناسبة فقط.
- ✓ قم بتدوير Refresh Token وإلقائه عند تسجيل الخروج.
- ✓ راقب حركة Tokens وسجل أي نشاط غير طبيعي.

ملاحظات



كل Flow يهدف للحصول على Access Token للوصول إلى الموارد المحمية.



مثال باستخدام قصير العمر و Refresh Refresh أطول عمراً ويُستخدم للتجديد.

مثال رأس الطلب (Authorization Header)

استخدم ال Access Token في كل طلب إلى API.

Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

مثال باستخدام curl

```
curl -H "Authorization: Bearer <ACCESS_TOKEN>" \
https://api.example.com/data
```

OAuth 2.0 يوفر طرق مرنة وآمنة للحصول على الوصول للموارد، اختر ال Flow المناسب بناءً على نوع تطبيقك واحتياجات الأمان.





المصادقة والتفويض JWT / OAuth

دليل مختصر لفهم المصادقة والتفويض في الأنظمة الحديثة



5. JWT Security

اتباع أفضل ممارسات الأمان عند استخدام JSON Web Tokens لحماية التطبيق والمستخدمين من الهجمات الشائعة.

ما هو هدف الأمان في JWT ؟

ضمان أن الـ Token صادر من جهة موثوقة، لم يتم التلاعب بمحتواه، ويستخدم بشكل آمن ولمدة زمنية مناسبة.

ممارسات أمان أساسية

1 توقيع قوي (Signature)

استخدم خوارزمية توقيع قوية مثل:

HS256 / RS256 / ES256



لا تستخدم خوارزميات ضعيفة (مثل: none, HS1)

2 انتهاء الصلاحية (exp)

حدد وقت انتهاء صلاحية قصير للـ Access Token.



تحقق دائماً من (exp claim) لرفض التوكن المنتهي.

3 المُصدر (iss)

حدد الجهة المصدرة للتوكن (Issuer).



تحقق من أن قيمة (iss) تطابق الجهة الموثوقة.

4 الجمهور (aud)

حدد الجمهور أو الخدمة المستهدفة (Audience).



تحقق من أن (aud) يطابق الخدمة التي تقبل التوكن.

5 عدم تخزين بيانات حساسة

لا تضع معلومات حساسة داخل الـ Payload مثل كلمات المرور أو بيانات البطاقة.



الـ Payload يمكن فك ترميزه (Base64) بسهولة.

6 استخدم HTTPS دائماً

أرسل الـ JWT فقط عبر اتصال مشفر (HTTPS).



يمنع اعتراض أو سرقة التوكن أثناء الإرسال.

7 Refresh Token بأمان

خزن الـ Refresh Token HttpOnly Cookie.



اجعله طويل العمر.

- دوره (Rotate) عند كل استخدام.
- أعد التوكن القديم غير صالح.

8 إبطال التوكن (Revocation)

وفر آلية لإبطال التوكن عند:



- تسجيل الخروج.
- تغيير كلمة المرور.
- اكتشاف نشاط مريب.
- إبطال الحساب.

هجمات شائعة وكيفية الوقاية منها



Token Theft

سرقة التوكن من جهاز المستخدم أو الشبكة.

الوقاية:

استخدم HTTPS و HttpOnly Cookie وقصر مدة التوكن.



XSS

هجوم نصوص خبيثة لسرقة التوكن من المتصفح.

الوقاية:

نوفرز HttpOnly Cookie و Content Security Policy.



CSRF

إجبار المستخدم على تنفيذ طلب غير مرغوب.

الوقاية:

استخدم SameSite Cookie و CSRF Token.



Replay Attack

إعادة استخدام توكن مسروق للوصول غير المصرح.

الوقاية:

استخدم التوكن قصير العمر وتحقق من (jti).



Weak Secret / Keys

استخدام مفاتيح ضعيفة أو تسريب المفاتيح.

الوقاية:

استخدم مفاتيح قوية وطويلة واحفظها بأمان.



Long-lived Tokens

توكنات طويلة العمر تزيد خطر الاختراق.

الوقاية:

اجعل الـ Access Token قصير العمر واستخدم Refresh Token.

قائمة التحقق من صحة JWT

(عند استقبال التوكن)



- تحقق من صحة التوقيع (Signature).
- تحقق من عدم انتهاء الصلاحية (exp).
- تحقق من المُصدر (iss).
- تحقق من الجمهور (aud).
- تحقق من عدم إبطال التوكن.
- تحقق من Claims الأخرى (مثل roles, scope).

تخزين التوكنات الموصى به



Access Token
قصير العمر
(5 - 15 دقيقة)

في الذاكرة فقط
أو في HttpOnly Secure Cookie



Refresh Token
طويل العمر
(أبام - أسابيع)

HttpOnly Secure Cookie (يفضل)



لا تستخدم

Local Storage
Session Storage
لحفظ التوكنات

مثال على Claims مهمة للأمان

```
{  "iss": "https://auth.example.com",  "sub": "user_12345",  "aud": "api.example.com",  "exp": 1710163200,  "nbf": 1710159600,  "iat": 1710159600,  "jti": "a1b2c3d4-e5f6-7890",  "scope": "read write",  "role": "Admin"}
```

كل Claims يجب التحقق منها على الخادم.

نصائح سريعة



اجعل الـ Access Token قصير العمر.



استخدم توقيع قوي ولا تكشف المفاتيح.



لا تثق في التوكن قبل التحقق من جميع الـ Claims.



دور Refresh Token وأعد التوكن القديم غير صالح.



راقب الأنشطة وسجل المحاولات المشبوهة.

الأمان لا يعتمد على التوكن فقط، بل على تصميم النظام وسلوك التطبيق والممارسات الصحيحة.





المصادقة والتفويض

JWT / OAuth

دليل مختصر لفهم المصادقة والتفويض في الأنظمة الحديثة



6. تخزين Tokens

كيفية تخزين Access Token و Refresh Token بطريقة آمنة للحماية من الهجمات الشائعة مثل XSS و CSRF و سرقة البيانات.

ما هو تخزين Token ؟

هو تحديد المكان والطريقة التي يتم بها حفظ Tokens على جهاز المستخدم أو في المتصفح ليتم استخدامها لاحقاً في طلبات المصادقة المصرح بها.

خيارات تخزين Tokens

1. HttpOnly Cookie



تخزين Token في Cookie مع تفعيل خاصية HttpOnly

المزايا

- ✓ حماية عالية من هجمات XSS
- ✓ يتم إرساله تلقائياً مع الطلبات
- ✓ مناسب لتطبيقات الويب

العيوب

- ✗ عرضة لهجمات CSRF
- ✗ لا يمكن الوصول إليه من JavaScript

2. Secure Cookie



تخزين Token في Cookie مع تفعيل خاصية Secure

المزايا

- ✓ يتم إرساله فقط عبر HTTPS
- ✓ يمنع التنصت أثناء النقل
- ✓ مستوى أمان أعلى

العيوب

- ✗ يتطلب HTTPS دائماً
- ✗ لا يمكن الوصول إليه من JavaScript

3. SameSite Cookie



تحديد سياسة SameSite في ملف تعريف الارتباط

SameSite

- ➔ **Strict**
لا يتم إرساله مع طلبات خارج الموقع.
- ➔ **Lax**
يُرسل مع التنقلات الآمنة مثل GET.
- ➔ **None**
يُرسل مع جميع الطلبات. Secure.

4. Local Storage



تخزين Token في Local Storage باستخدام JavaScript

المزايا

- ✓ سعة تخزين كبيرة
- ✓ سهل الوصول عبر JavaScript
- ✓ مناسب لتطبيقات SPA

العيوب

- ✗ عرضة لهجمات XSS
- ✗ يستمر حتى بعد إغلاق المتصفح

5. Session Storage



تخزين Token في Session Storage باستخدام JavaScript

المزايا

- ✓ يعمل فقط ضمن تبويب واحد
- ✓ يتم حذفه عند إغلاق التاب
- ✓ أكثر أماناً من Local Storage

العيوب

- ✗ عرضة لهجمات XSS
- ✗ سعة تخزين محدودة

مقارنة سريعة

طريقة التخزين	مناح JavaScript	مقاوم لهجمات XSS	مقاوم لهجمات CSRF	يُرسل مع الطلبات تلقائياً	HTTPS	يُحذف عند إغلاق المتصفح
1. HttpOnly Cookie	✗ لا	✓ نعم	لا (إلا مع SameSite)	✓ نعم	✓ موصى به	✗
2. Secure Cookie	✗ لا	✓ نعم	لا (إلا مع SameSite)	✓ نعم	✓ نعم (الزامي)	✗
3. SameSite Cookie	✗ لا	✓ نعم	✓ نعم (حسب القيمة)	✓ نعم	✓ موصى به	✗
4. Local Storage	✓ نعم	✗ لا	✗	✗	✓ موصى به	✗
5. Session Storage	✓ نعم	✗ لا	✗	✗	✓ موصى به	✓ نعم

التوصيات



تطبيقات الويب التقليدية

استخدم HttpOnly Cookie
SameSite=Lax و Secure.
لأمان أفضل ضد XSS و CSRF.



تطبيقات SPA

Memory Storage أو Session Storage مع
HttpOnly في Refresh Token
Cookie إن أمكن.



تطبيقات الجوال

احفظ Tokens في Secure Storage الخاص بالنظام.
مثل Keychain (iOS) أو Keystore (Android).



ممارسات عامة

- لا تخزن Tokens في Secure بدون سبب ضروري.
- اجعل Access Token قصير العمر.
- استخدم Refresh Token لتجديد الجلسة.
- فعل HTTPS دائماً.

أفضل الممارسات



لا تعرض Token في عناوين URL.



Refresh Token بتدوير دوري.



اجعل مدة صلاحية Access Token قصيرة.



راقب الأنشطة المشبوهة وسجل الأحداث.



ألغِ Token عند تسجيل الخروج.



اختيار طريقة التخزين المناسبة يعتمد على نوع التطبيق ومتطلبات الأمان.





المصادقة والتفويض JWT / OAuth

دليل مختصر لفهم المصادقة والتفويض في الأنظمة الحديثة



دورة حياة Token

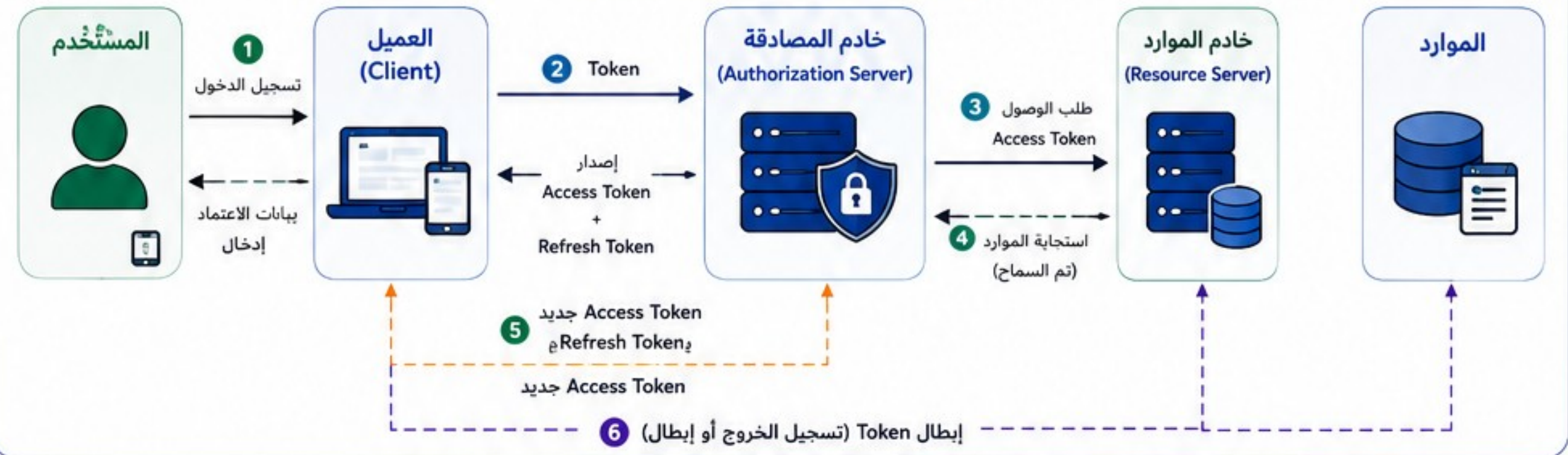
تمثل دورة حياة الـ Token المراحل التي يمر بها منذ إصدارها حتى انتهاء صلاحيتها أو إبطالها.

ما هي دورة حياة Token ؟

هي سلسلة من الخطوات تبدأ بتسجيل الدخول وإصدار Token وتنتهي بانتهائها أو إبطالها، لضمان الوصول الآمن والمرن للموارد.



تدفق دورة حياة Token



مكونات الـ Token



أفضل الممارسات

- ✓ استخدم Access Token قصير العمر.
- ✓ خزن Refresh Token في مكان (HttpOnly Cookie).
- ✓ فعل HTTPS دائماً لحماية النقل.
- ✓ تحقق من جميع Claims في الخادم.
- ✓ أعد تدوير Refresh Token بشكل دوري.
- ✓ أطبق إبطال Token عند تسجيل الخروج أو الاشتباه في اختراق.

مثال زمني لدورة حياة Token



إدارة دورة حياة الـ Token بشكل صحيح تضمن أمان النظام ومرونة تجربة المستخدم.





المصادقة والتفويض JWT / OAuth

دليل مختصر لفهم المصادقة والتفويض في الأنظمة الحديثة



8. أخطاء ومخاطر شائعة

تجنب هذه الأخطاء لحماية نظامك وضمان أمان المصادقة والتفويض.

ما الهدف من معرفة الأخطاء؟

فهم الأخطاء والمخاطر الشائعة يساعد على بناء تطبيقات آمنة، وحماية البيانات، ومنع الوصول غير المصرح به.

أخطاء ومخاطر شائعة في استخدام JWT

1 تخزين Token في مكان غير آمن



تخزين الـ Token في LocalStorage أو في الكود يعرضه للسرقة بسهولة عبر XSS.

الوقاية

- استخدم HttpOnly Cookie.
- لا تخزن الـ Token في الكود.

2 إهمال التحقق من التوقيع (Signature)



عدم التحقق من صحة توقيع الـ JWT يسمح بتزوير الـ Token.

الوقاية

- تحقق من التوقيع باستخدام المفتاح العام الصحيح.
- لا تعتمد فقط على فك الـ Token دون التحقق.

3 استخدام خوارزمية ضعيفة



استخدام خوارزميات ضعيفة مثل HS256 مع مفاتيح قصيرة أو غير آمنة.

الوقاية

- استخدم خوارزميات قوية مثل RS256 أو ES256.
- استخدم مفاتيح قوية وآمنة.

4 إهمال التحقق من الادعاءات (Claims)



عدم التحقق من exp, iss, aud, nbf, sub قد يسمح باستخدام Token غير صالح.

الوقاية

- تحقق من جميع الادعاءات دائماً.
- تأكد من انتهاء الصلاحية (exp).

5 صلاحيات زائدة في الـ Token



منح صلاحيات أكثر من اللازم يعرض البيانات والعمليات للمخاطر.

الوقاية

- منح أقل صلاحيات ممكنة.
- استخدم مبدأ أقل امتياز.

أخطاء ومخاطر شائعة في استخدام OAuth 2.0

1 اختيار Flow غير مناسب



استخدام Flow غير مناسب قد يسمح بتسرب الـ Token.

الوقاية

- اختر Flow المناسب لكل تطبيق (Authorization Code, PKCE, Client Credentials...).

2 عدم استخدام PKCE في التطبيقات العامة



عدم استخدام PKCE في تطبيقات الويب والموبايل يزيد من خطر سرقة Authorization Code.

الوقاية

- استخدم PKCE دائماً في التطبيقات العامة (Public Clients).

3 حفظ Refresh Token بشكل غير آمن



تخزين Refresh Token في أماكن غير آمنة يتيح للمهاجم الحصول على وصول دائم.

الوقاية

- احفظ Refresh Token في HttpOnly Cookie، أو خزنه مشفراً في الآلة الآمنة.

4 نطاق صلاحيات واسع (Scopes)



طلب Scopes أكثر من اللازم يعطي التطبيق وصولاً غير مطلوب.

الوقاية

- اطلب Scopes الضرورية فقط.
- راجع النطاقات بشكل دوري.

5 عدم التحقق من هوية المزود (Issuer)



عدم التحقق من الـ Issuer قد يسمح بالاعتماد على Tokens صادرة من جهة غير موثوقة.

الوقاية

- تحقق من الـ Issuer (iss) ومطابقته للمزود الموثوق.

توصيات عامة للأمان



استخدم HTTPS دائماً لتشفير البيانات أثناء النقل.



قم بتدوير المفاتيح بشكل دوري.



اجعل مدة صلاحية الـ Access Token قصيرة.



استخدم Refresh Token للحصول على وصول مستمر بدلاً من Access Token طويلة.



سجل الأحداث والشذوذ وراقب محاولات الوصول غير المصرح بها.



لا تنقل بيانات حساسة داخل الـ Token، بل استخدم معرفات فقط.

قائمة مراجعة سريعة

JWT

- هل يتم التحقق من توقيع الـ Token؟
- هل يتم التحقق من الادعاءات (exp, iss, aud, nbf, sub)؟
- هل مدة صلاحية الـ Token مناسبة وقصيرة؟
- هل الـ Token مخزن في مكان آمن (HttpOnly Cookie)؟
- هل الخوارزمية والمفاتيح قوية وآمنة؟
- هل الصلاحيات داخل الـ Token هي الأقل ما يمكن؟



OAuth 2.0

- هل يتم اختيار الـ Flow المناسب للتطبيق؟
- هل يتم استخدام PKCE في التطبيقات العامة؟
- هل الـ Scopes المطلوبة هي الضرورية فقط؟
- هل الـ Refresh Token مخزن بشكل آمن؟
- هل يتم التحقق من هوية المزود (Issuer)؟
- هل يتم إبطال الـ Token عند تسجيل الخروج أو الاشتباه؟



تذكر

الأمان عملية مستمرة وليست مهمة تُنفذ مرة واحدة، راجع إعدادات المصادقة والتفويض بانتظام وابق على اطلاع على أفضل الممارسات.





حماية الـ API من الوصول غير المصرح به، وتحديد هوية المستخدم وصلاحياته، والسماح بالوصول إلى الموارد المناسبة فقط.

رمز مميز يحمل هوية المستخدم
وصلاحياته (Scopes/Claims).

The diagram illustrates the OAuth 2.0 authorization flow involving three main components: the Client (عميل), the Authorization Server (خادم المصادقة), and the Resource Server (خادم ال API).

- Client Request:** The Client sends a request to the Authorization Server to request authorization (طلب التفويض (الموافقة)).
- User Login:** The Authorization Server prompts the user for login (تسجيل الدخول (من المستخدم)).
- Granting Token:** The Authorization Server grants a token to the Client (منح التفويض (Coda ثم موزمبيم)).
- Access Token:** The Client sends the Access Token to the Resource Server (استبدال الكود بـ Access Token).
- Access Token (JWT):** The Client sends the Access Token (JWT) to the Resource Server.
- Token Verification:** The Resource Server verifies the token and checks the user's permissions (التحقق من الـ Token والصلاحيات).
- Data Access:** The Resource Server provides access to the requested data (الوصول للبيانات).
- Token Refresh:** The Client requests a new token from the Authorization Server (الاستجابة (البيانات)).

sub	هوية المستخدم (ID)
name	اسم المستخدم
role	الدور (Role)
scopes	الصلاحيات (Scopes)
iat	وقت الإصدار
exp	وقت الانتهاء
iss	الجهة المصدرة للـ Token
aud	الجهة المستهدفة بالـ Token

طبة، مبدأ أقل، صلاحية (Least Privilege).

OAuth 2.0	JWT	الجانِب
تقويض الوصول للموارد	تمثيل هوية وصلاحيات	الهدف
التطبيقات الخارجية	ال APIs الخاصة	الاستخدام
Access Token	Access Token (JWT)	الناتج الأساسي
معيّار تقويض	معيّار ترميز (Token)	الحالة
لا (منفصل عن ال Token)	نعم (داخل ال Token)	يحمل الصلاحيات؟

- تخزين الـ Token في أماكن غير آمنة.



أمانك... أولويتنا



المصادقة والتفويض JWT / OAuth



دليل مختصر لفهم المصادقة والتفويض في الأنظمة الحديثة

10. JWT / OAuth في ASP.NET Core

كيفية تطبيق المصادقة والتفويض باستخدام API و OAuth في تطبيقات ASP.NET Core بطريقة آمنة ومنظمة.

لماذا نستخدمه؟

تأمين تطبيقات ASP.NET Core باستخدام JWT للمصادقة و OAuth 2.0 للتفويض والوصول إلى الموارد بشكل آمن ومنظم.

1 المتطلبات الأساسية



.NET
8 / 9
SDK



Visual Studio
2022
أو VS Code



فهم أساسيات
ASP.NET Core



معرفة
JWT و
OAuth 2.0

2 تثبيت الحزم المطلوبة



```
dotnet add package  
Microsoft.AspNetCore.Authentication.JwtBearer
```

دعم JWT في ASP.NET Core

يتضمن أدوات التحقق من التوقيع والترميز



3 JWT في Program.cs

```
var builder = WebApplication.CreateBuilder(args);  
builder.Services.AddAuthentication(options =>  
{  
    options.DefaultAuthenticateScheme =  
        JwtBearerDefaults.AuthenticationScheme;  
    options.DefaultChallengeScheme =  
        JwtBearerDefaults.AuthenticationScheme;  
})  
.AddJwtBearer(options =>  
    options.TokenValidationParameters = new  
        TokenValidationParameters  
    {  
        ValidateIssuer = true,  
        ValidateAudience = true,  
        ValidateLifetime = true,  
        ValidateIssuerSigningKey = true,  
        ValidIssuer = builder.Configuration["Jwt:Issuer"],  
        ValidAudience = builder.Configuration["Jwt:Audience"],  
        IssuerSigningKey = new SymmetricSecurityKey(  
            Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"])))  
    });  
builder.Services.AddAuthorization();  
var app = builder.Build();  
app.UseAuthentication();  
app.UseAuthorization();  
app.MapControllers();
```

C#

4 إصدار Token عند تسجيل الدخول

```
var claims = new[]  
{  
    new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),  
    new Claim(ClaimTypes.Name, user.UserName),  
    new Claim(ClaimTypes.Role, "Admin")  
};  
var key = new SymmetricSecurityKey(  
    Encoding.UTF8.GetBytes(configuration["Jwt:Key"]));  
var token = new JwtSecurityToken(  
    issuer: configuration["Jwt:Issuer"],  
    audience: configuration["Jwt:Audience"],  
    claims: claims,  
    expires: DateTime.UtcNow.AddHours(1),  
    signingCredentials: new SigningCredentials(key,  
        SecurityAlgorithms.HmacSha256)  
);  
var tokenString = new JwtSecurityTokenHandler()  
    .WriteToken(token);  
return Ok(new { token = tokenString });
```



5 نقاط النهاية (Controllers)



```
[Authorize]  
[ApiController]  
[Route("api/[controller]")]  
public class OrdersController : ControllerBase  
{  
    [HttpGet]  
    public IActionResult Get()  
    {  
        return Ok("مرحبًا بك في مرصفتي");  
    }  
    [Authorize(Roles = "Admin")]  
    [HttpDelete("{id}")]  
    public IActionResult Delete(int id)  
    {  
        return Ok("تم الحذف");  
    }  
}
```

استخدم [Authorize] لحماية أي نقطة نهاية ويمكن تحديد الأدوار أو الصلاحيات.

8 أفضل الممارسات

- استخدم HTTPS دائماً.
- أخزن Token في مكان آمن.
- إجعل مدة صلاحية Access Token قصيرة.
- استخدم Refresh Token لإصدار Token جديد.
- طبق سياسات التفويض (Roles / Claims).
- لا تخزن معلومات حساسة داخل Token.
- راقب السجلات والأنشطة المشبوهة.
- استخدم أقل الصلاحيات (Least Privilege).

9 إعدادات التطبيق (appsettings.json)

```
{  
  "Jwt": {  
    "Key": "your-very-strong-secret-key-1234567890",  
    "Issuer": "https://your-app.com",  
    "Audience": "your-client-app",  
    "ExpireHours": 1  
  },  
  "Authentication": {  
    "Google": {  
      "ClientId": "YOUR_GOOGLE_CLIENT_ID",  
      "ClientSecret": "YOUR_GOOGLE_CLIENT_SECRET"  
    }  
  }  
}
```

10 أخطاء شائعة

- إرسال ال Token في مكان غير صحيح.
- عدم التحقق من توقيع ال JWT.
- إهمال التحقق من انتهاء صلاحية ال Token.
- منح صلاحيات زائدة في ال Scopes.
- تخزين ال Token في أماكن غير آمنة.
- تعطيل التحقق من HTTPS.
- عدم تدوير المفاتيح بشكل دوري.

ملخص سريع



JWT

يستخدم للمصادقة بين التطبيق والخادم وتبادل المعلومات بطريقة آمنة بدون حالة.



OAuth 2.0

يستخدم لتفويض الوصول إلى الموارد نيابة عن المستخدم من مزود خارجي.



ASP.NET Core

يوفر أدوات قوية لتطبيق OAuth و JWT بسهولة وأمان.



الهدف

تأمين التطبيقات وضمان المستخدمين المصرح لهم فقط إلى الموارد الصحيحة.



النتيجة

نظام آمن، قابل للتوسع، ومتوافق مع أفضل المعايير الحديثة.

تذكر دائماً

الأمان ليس ميزة إضافية، بل هو أساس أي نظام ناجح وموثوق.

المصادقة والتفويض الصحيح أساس أمان تطبيقاتك وحماية بياناتك مستخدميك.



المصادقة والتفويض JWT / OAuth

دليل مختصر لفهم المصادقة والتفويض في الأنظمة الحديثة



11. مصطلحات سريعة

أهم المصطلحات التي تساعدك على فهم المصادقة والتفويض باستخدام JWT و OAuth

ما الهدف؟

فهم المصطلحات الأساسية لتسهيل العمل مع تقنيات المصادقة والتفويض.

الأيقونة	التعريف	الإنجليزية	المصطلح
	عملية التحقق من هوية المستخدم للتأكد من أنه الشخص الصحيح الذي يدعي أنه هو.	Authentication	المصادقة
	عملية تحديد ما إذا كان المستخدم الموثق مسموحاً له بالوصول إلى مورد معين أو تنفيذ إجراء معين.	Authorization	التفويض
	بيانات مشفرة تمثل تصريح الوصول، تُستخدم للتحقق من الهوية والصلاحيات.	Token	رمز الوصول
	اختصار لـ JSON Web Token، معيار مفتوح لتبادل المعلومات بأمان بين طرفين على شكل JSON.	JSON Web Token (JWT)	رمز ويب JSON
	إطار عمل تفويض يسمح للتطبيقات بالوصول إلى موارد المستخدم دون مشاركة بيانات الاعتماد الخاصة به.	OAuth 2.0	أو ثوث 2.0
	رمز قصير العمر يُستخدم للوصول إلى الموارد المحمية.	Access Token	رمز الوصول
	رمز يُستخدم للحصول على رمز وصول جديد عند انتهاء صلاحية الرمز الحالي.	Refresh Token	رمز التحديث
	مجموعة من بيانات الاعتماد (مثل اسم المستخدم وكلمة المرور) تُستخدم لتسجيل الدخول.	Credentials	بيانات الاعتماد
	الجهة التي تتحقق من هوية المستخدم وتصدر رموز الوصول.	Authorization Server	خادم التفويض
	التطبيق الذي يطلب الوصول إلى الموارد المحمية نيابة عن المستخدم.	Client Application	تطبيق العميل
	النظام الذي يحتوي على الموارد المحمية التي يتم الوصول إليها باستخدام الرموز.	Resource Server	خادم الموارد
	بيانات حول المستخدم ضمن الـ JWT، مثل الهوية، الصلاحيات، والدور.	Claims	الادعاءات
	المتغيرات التي تحدد فترة صلاحية الرمز وتوقيته.	Expiration	مدة الصلاحية
	توقيع رقمي يُستخدم للتحقق من سلامة الرمز وعدم التلاعب به.	Signature	التوقيع

أنواع الرموز

- Access Token**
للوصول إلى الموارد المحمية.
- Refresh Token**
للحصول على رمز وصول جديد.
- ID Token (في OAuth)**
يحتوي على معلومات عن هوية المستخدم.

تذكر دائماً

- استخدم HTTPS دائماً.
- لا تخزن الرموز في أماكن غير آمنة.
- تحقق من صلاحيات المستخدم دائماً.
- أقل صلاحيات ممكنة (Least Privilege).
- راقب انتهاء صلاحية الرموز وحديثها.

فروق سريعة

JWT	OAuth 2.0
تنسيق رموز (Token) لتمثيل الهوية والبيانات.	إطار عمل تفويض للوصول إلى الموارد.
يمكن أن يعمل بشكل مستقل (Self-contained).	يعتمد على خادم تفويض (Authorization Server).
يستخدم للتصديق (Authentication) وتبادل المعلومات.	يستخدم للتفويض (Authorization).



فهم هذه المصطلحات هو أساس بناء أنظمة آمنة وموثوقة باستخدام JWT و OAuth.





المصادقة والتفويض JWT / OAuth

دليل مختصر لفهم المصادقة والتفويض في الأنظمة الحديثة



TOKEN



12. مثال معماري مختصر

يوضح هذا المثال كيف تعمل المصادقة والتفويض باستخدام OAuth 2.0 و JWT في تطبيق ويب حديث.

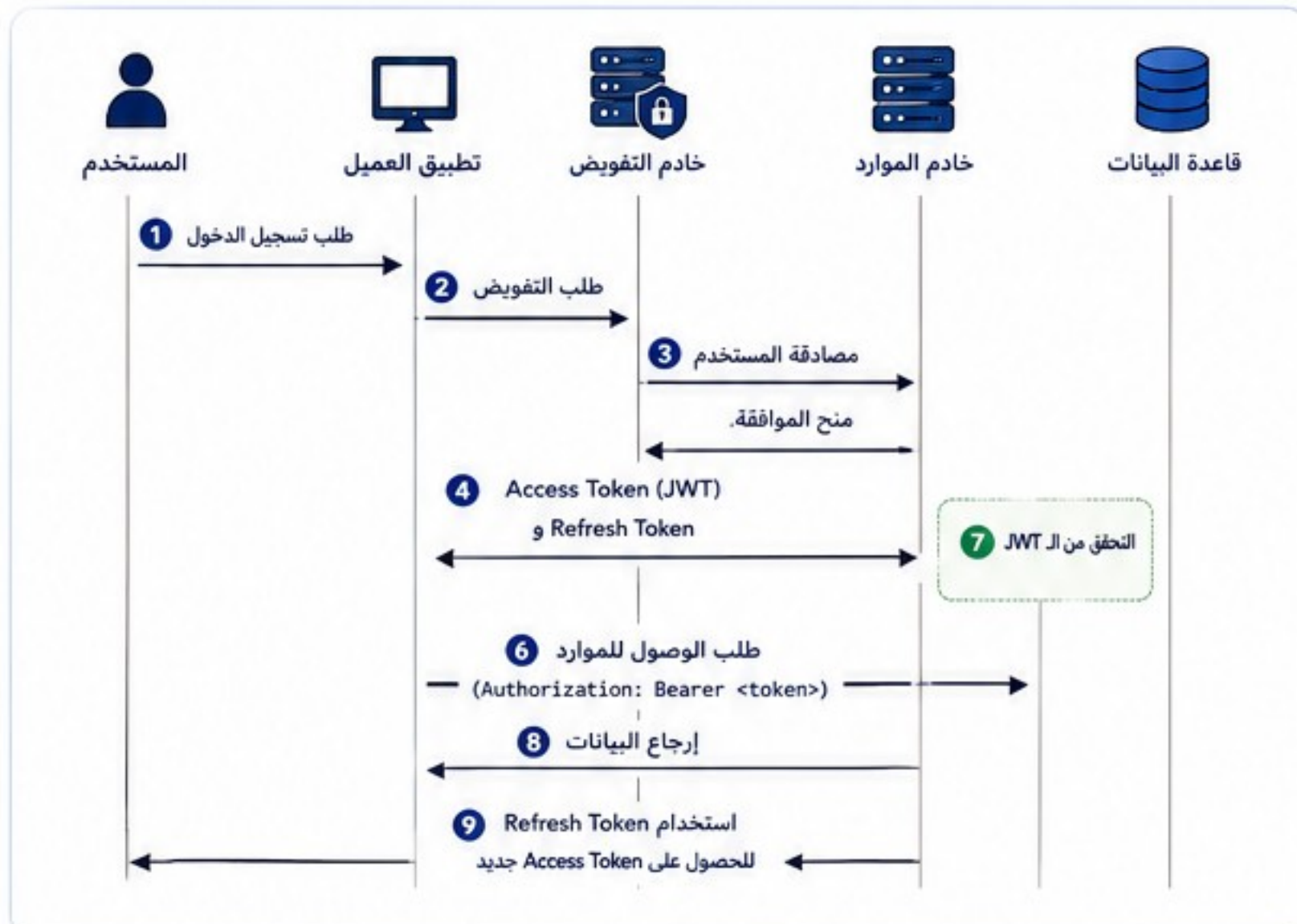
النموذج العام

المستخدم يطلب الوصول إلى تطبيق محمي.
يتم استخدام OAuth 2.0 للحصول على Token.
يتم استخدام JWT للوصول إلى الموارد المحمية.



تدفق العمل (تسلسل الخطوات)

- 1 يطلب المستخدم تسجيل الدخول من تطبيق العميل.
- 2 يتم تحويل المستخدم إلى خادم التفويض (Authorization).
- 3 يقوم المستخدم بالمصادقة، ويمنح الموافقة (إن لزم).
- 4 يصدر خادم التفويض Refresh Token (JWT) Access Token.
- 5 يخزن تطبيق العميل الـ Access Token (عادة في الذاكرة أو التخزين الآمن).
- 6 يرسل تطبيق العميل الـ Access Token في كل طلب إلى خادم الموارد ضمن ترويسة Authorization.
- 7 يتحقق خادم الموارد من صحة الـ JWT (التوقيع، الصلاحية، الجهة المصدرة، الجمهور...).
- 8 إذا كان الـ JWT صالحاً، يتم الوصول إلى الموارد المطلوبة من قاعدة البيانات.
- 9 تُستخدم Refresh Token للحصول على Access Token جديد انتهاءً.



مثال طلب HTTP

طلب إلى خادم الموارد:

```

GET /api/data HTTP/1.1
Host: api.example.com
Authorization: Bearer <ACCESS_TOKEN>
Accept: application/json
  
```

استجابة من الخادم:

```

{
  "data": "القيم المطلوبة"
  "status": "success"
}
  
```

محتوى الـ JWT (مثال)

Header	Payload (Claims)	Signature
{ "alg": "HS256", "typ": "JWT" }	{ "sub": "12345", "name": "أحمد", "role": "User", "exp": 1718800000, "iss": "auth.server" }	HMACSHA256(base64UrlEncode (header) + "." + base64UrlEncode (payload), secret)

الترميز النهائي:

xxxxxx.yyyyyy.zzzzz

فوائد هذا النموذج

- ✓ أمان عالي باستخدام JWT (توقيع رقمي).
- ✓ فصل المسؤوليات بين التفويض والموارد.
- ✓ قابلية التوسع ودعم التطبيقات المتعددة.
- ✓ تقليل الحمل على خادم التفويض.
- ✓ دعم الوصول إلى APIs بشكل آمن.

أفضل الممارسات

- ✓ استخدم HTTPS دائماً.
- ✓ اجعل مدة صلاحية Access Token قصيرة.
- ✓ خزن Tokens في مكان آمن.
- ✓ تحقق من الجمهور (aud) والجهة المصدرة (iss).
- ✓ أدر مفاتيح التوقيع بعناية.

أنواع Tokens في هذا النموذج

- Access Token (JWT): يُستخدم للوصول إلى الموارد.
- Refresh Token: يُستخدم للحصول على Access Token جديد.

تذكر دائماً

الأمان ليس ميزة إضافية، بل هو أساس أي نظام ناجح وموثوق.

فهم هذا النموذج يساعدك على بناء أنظمة آمنة وموثوقة وقابلة للتوسع.